Laboratorio: gestión completa CRUD con roles (Supabase)

App `dam-crm-app` · Ionic 9 + Angular 22 standalone · Supabase (Auth + PostgreSQL + RLS)
Jorge González Gómez · 2º DAM · Desarrollo de Interfaces (0488) · Universidad Nebrija · versión 2, 26-10-2026
Código: github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/06-Supabase-Auth-Roles

Práctica completa. Hay login con Supabase y dos roles: el **admin** crea, edita y borra clientes, y el **user** solo los consulta. La seguridad la impone la base de datos (RLS), no solo la interfaz. Todo el código está comentado en GitHub.

App general de repaso. Para entender bien los conceptos de las prácticas 04–08 sin rehacerlas una a una, estoy haciendo una sola app de gestión con roles que junta lo importante de cada una (Supabase, guards, formularios reactivos y estado con signals). Está en el mismo repositorio: rama `app-gestion-roles`, carpeta `09-App-Gestion-Roles`.

1. Backend en Supabase

Proyecto `dam-crm` (West EU). En el SQL Editor ejecuté el script del laboratorio: enum `user_role` (`admin` / `user`), tabla `profiles` vinculada a `auth.users`, tabla `clients` (nombre, empresa, email único) y RLS en las dos. Añadí los `GRANT` para que la Data API pueda usar las tablas.

Política RLS	Operación	Quién puede
Leer perfil propio (`profiles`)	SELECT	Cada usuario, solo su fila: `auth.uid() = id`
Lectura para autenticados (`clients`)	SELECT	Cualquier usuario con sesión
Escritura solo para admins (`clients`)	INSERT · UPDATE · DELETE	Solo si en `profiles` su rol es `admin` (USING + WITH CHECK)

Creé dos usuarios en Auth (`admin@dam.com` y `user@dam.com`, autoconfirmados) y les di su rol insertando su fila en `profiles`:

```
INSERT INTO public.profiles (id, role) SELECT id, 'admin' FROM auth.users WHERE email = 'admin@dam.com';
INSERT INTO public.profiles (id, role) SELECT id, 'user'  FROM auth.users WHERE email = 'user@dam.com';
```

2. Estructura del proyecto Ionic

```
dam-crm-app/src/
|-- environments/environment.ts       supabaseUrl + supabaseKey (clave pública publishable)
`-- app/
    |-- models/client.model.ts        interfaz Client (id?, nombre, empresa, email, created_at?)
    |-- services/supabase.service.ts  login/logout, loadRole(), CRUD · signals de usuario y rol
    |-- guards/auth.guard.ts          sin sesión -> /login
    |-- login/                        pantalla de acceso
    `-- home/                         listado + formulario (admin) + ion-item-sliding (admin)
```

3. Estado global con signals y UI según el rol

El servicio guarda quién está conectado y su rol en signals. La página los lee, así que al iniciar o cerrar sesión la interfaz cambia sola:

```
// supabase.service.ts
public currentUser = signal<User | null>(null);
public userRole    = signal<'admin' | 'user' | null>(null);
public isAdmin     = computed(() => this.userRole() === 'admin');   // se recalcula solo

async login(email: string, password: string) {
  const { data, error } = await this.supabase.auth.signInWithPassword({ email, password });
  if (error) throw error;
  this.currentUser.set(data.user);
  await this.loadRole(data.user.id);       // profiles.select('role').eq('id', ...).single()
}
```

```
<!-- home.page.html -->
@if (isAdmin()) { <ion-card> formulario reactivo Nuevo / Editar cliente </ion-card> }
@else           { <ion-card> Modo lectura </ion-card> }

@for (client of clients(); track client.id) {
  <ion-item-sliding #sliding>
    <ion-item> {{ client.nombre }} · {{ client.empresa }} | {{ client.email }} </ion-item>
    @if (isAdmin()) { <ion-item-options side="end"> editar · borrar </ion-item-options> }
  </ion-item-sliding>
} @empty { No hay clientes registrados. }
```

El formulario usa `FormBuilder` con las validaciones del enunciado (nombre obligatorio y mínimo 3, empresa obligatoria, email obligatorio y válido) y `errorText` en cada campo. El diseño usa `ion-list` dentro de un `ion-grid`: una columna en el móvil, y formulario y lista en dos columnas en tablet o escritorio.

4. Cambios respecto al código del enunciado

Enunciado	Lo que hice y por qué
Asume que el usuario «ya está logueado»	Añadí pantalla de login y un guard: sin sesión, la política «Lectura para autenticados» no devuelve nada.
Estado en propiedades normales (`clients: Client[]`, `isToastOpen`…)	Lo pasé a signals: el proyecto usa Angular 22 sin zone.js y la vista no se actualizaba tras los `await`.
Imports desde `@ionic/angular/standalone`	Desde `@ionic/angular`: en Ionic 9 los componentes standalone están en el paquete principal.
`ion-item-option (click)`	Cierro el item con `sliding.close()` antes de editar o borrar.
Errores en inglés (`e.message`)	`textoError()` los traduce: email repetido (código 23505), sin permiso (42501), credenciales incorrectas.

5. Comprobaciones

Acción	admin	user	Quién lo controla
Ver el listado de clientes	Sí	Sí	RLS «Lectura para autenticados»
Crear / editar / borrar	Sí	No	RLS «Escritura solo para admins» (y la UI lo oculta con `@if`)
Entrar en `/home` sin sesión	Redirige a `/login`		`auth.guard.ts`

`ng build` sin errores · `ng lint`: all files pass · `ng test`: tests OK.

6. Ejecutar

En `06-Supabase-Auth-Roles/dam-crm-app`: `npm install` y `ionic serve`. Usuarios de prueba: `admin@dam.com` (admin) y `user@dam.com` (user).